山东大学 计算机科学与技术 学院

自然语言处理 课程实验报告

学号：202200130077	姓名： 薛润泽	班级： 智能 22

实验题目： 实验四：命名实体识别

实验内容：

1.利用 Chinese.txt 和 English.txt 的中英文句子，在实验二的基础上，继续利用以下给定的中英文工具进行命名实体识别。并对不同工具产生的结果进行简要对比分析，将实验过程与结果写成实验报告，实验课结束后提交。

2.使用 BERT + Bi-LSTM + CRF 实践命名实体识别

实验过程中遇到和解决的问题：

（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

实验没有提供训练集，使用了 CLUENER 细粒度命名实体识别数据集作为训练集

实验步骤：

1、配置英文命名实体识别环境 Nltk, Spacy, stanza

2、配置中文命名实体识别环境 Jieba, stanza，SnowNLP, THULAC, NLPIR

3、使用 BERT + Bi-LSTM + CRF 实践命名实体识别

以中文命名实体识别为例，实践 BERT + Bi-LSTM + CRF 命名实体识别流程。

（1）使用 CLUENER 细粒度命名实体识别数据集作为训练集

https://github.com/CLUEbenchmark/CLUENER2020

（2）模型训练流程，首先输入训练集的输入文本序列；BERT Tokenizer 进行分词和 ID 转化；BERT 模型提取上下文嵌入；Dropout 防止过拟合；BI-LSTM 层捕捉序列依赖；Dropout 防止过拟合；接入一个全连接层；CRF 层进行序列解码

实验结果：

1、英文命名实体识别结果：

（1）Nltk 结果

```
(nlp) D:\大学\大三下\自然语言处理\实验\exp4>python ex4.py
NLTK NER结果：
(S
  Xi/NN
  Jinping/NNP
  ,/,
  male/NN
  ,/,
  (PERSON Han/NNP)
  ethnicity/NN
  ,/,
  was/VBD
  born/VBN
  in/IN
  June/NNP
  1953/CD
  and/CC
  is/VBZ
  from/IN
  Fuping/VBG
  ,/,
  (PERSON Shaanxi/NNP Province/NNP)
  ./.
  He/PRP
  began/VBD
  his/PRP$
  first/JJ
  job/NN
  in/IN
  January/NNP
  1969/CD
  and/CC
  joined/VBD
  the/DT
  (ORGANIZATION Communist/NNP Party/NNP)
  of/IN
  (GPE China/NNP)
  (/(
  (ORGANIZATION CPC/NNP)
  )/)
  in/IN
  January/NNP
  1974/CD
  ./.
  Xi/VB
  graduated/VBN
  from/IN
  (ORGANIZATION School/NNP)
  of/IN
```

```
(/(
(ORGANIZATION CPC/NNP)
)/)
in/IN
January/NNP
1974/CD
./.
Xi/VB
graduated/VBN
from/IN
(ORGANIZATION School/NNP)
of/IN
(ORGANIZATION Humanities/NNP)
and/CC
(ORGANIZATION Social/NNP Sciences/NNPS)
,/,
(ORGANIZATION Tsinghua/NNP University/NNP)
where/WRB
he/PRP
completed/VBD
an/DT
in-service/JJ
graduate/NN
program/NN
in/IN
(GPE Marxist/NNP)
theory/NN
and/CC
ideological/JJ
and/CC
political/JJ
education/NN
./.
He/PRP
holds/VBZ
a/DT
Doctor/NNP
of/IN
(GPE Law/NNP)
degree/NN
./.
Xi/NN
is/VBZ
currently/RB
(ORGANIZATION General/NNP)
Secretary/NNP
of/IN
the/DT
(ORGANIZATION CPC/NNP Central/NNP Committee/NNP)
,/,
Chairman/NNP
```

```
is/VBZ
currently/RB
(ORGANIZATION General/NNP)
Secretary/NNP
of/IN
the/DT
(ORGANIZATION CPC/NNP Central/NNP Committee/NNP)
,/,
Chairman/NNP
of/IN
the/DT
(ORGANIZATION CPC/NNP Central/NNP)
Military/NNP
Commission/NNP
,/,
President/NNP
of/IN
the/DT
(ORGANIZATION People/NNP)
's/POS
(ORGANIZATION Republic/NNP)
of/IN
(GPE China/NNP)
(/(
(ORGANIZATION PRC/NNP)
)/)
,/,
and/CC
Chairman/NNP
of/IN
the/DT
(ORGANIZATION PRC/NNP Central/NNP)
Military/NNP
Commission/NNP
./.)
```

（2）Spacy 命名实体识别结果

```
(spacy) D:\大学\大三下\自然语言处理\实验\exp4>python ex4.py

Spacy NER结果：
[('Xi Jinping', 'PERSON'), ('Han', 'NORP'), ('June 1953', 'DATE'), ('Fuping', 'GPE'), ('Shaanxi Province', 'GPE'), ('fir
st', 'ORDINAL'), ('January 1969', 'DATE'), ('the Communist Party of China', 'ORG'), ('CPC', 'ORG'), ('January 1974', 'DA
TE'), ('Xi', 'PERSON'), ('School of Humanities and Social Sciences', 'ORG'), ('Tsinghua University', 'ORG'), ('Marxist',
 'NORP'), ('Xi', 'PERSON'), ('the CPC Central Committee', 'ORG'), ('the CPC Central Military Commission', 'ORG'), ("the
People's Republic of China", 'GPE'), ('PRC', 'GPE'), ('the PRC Central Military Commission', 'ORG')]

(spacy) D:\大学\大三下\自然语言处理\实验\exp4>
```

（3）stanza 命名实体识别结果

```
Stanza (StanfordNLP) NER结果：
[('Xi Jinping', 'PERSON'), ('Han', 'NORP'), ('June 1953', 'DATE'), ('Fuping, Shaanxi Province', 'GPE'), ('first', 'ORDIN
AL'), ('January 1969', 'DATE'), ('the Communist Party of China', 'ORG'), ('CPC', 'ORG'), ('January 1974', 'DATE'), ('Xi'
, 'PERSON'), ('School of Humanities and Social Sciences', 'ORG'), ('Tsinghua University', 'ORG'), ('Marxist', 'NORP'), (
'a Doctor of Law degree', 'WORK_OF_ART'), ('Xi', 'PERSON'), ('the CPC Central Committee', 'ORG'), ('the CPC Central Mili
tary Commission', 'ORG'), ("the People's Republic of China", 'ORG'), ('PRC', 'GPE'), ('the PRC Central Military Commissi
on', 'ORG')]
```

2、中文命名实体识别结果：

（1）jieba 命名实体识别结果

```
Jieba NER结果：
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\薛润泽\AppData\Local\Temp\jieba.cache
Loading model cost 0.367 seconds.
Prefix dict has been built successfully.
[('3', 'm'), ('月', 'm'), ('23', 'm'), ('日', 'm'), ('下午', 't'), (', ', 'x'), ('“', 'x'), ('青年报', 'n'), ('杯赛', 'n'), ('”', 'x'), ('U19', 'eng'), ('邀请赛', 'n'), ('在', 'p'), ('越南', 'ns'), ('芽庄', 'n'), ('进行', 'v'), (', ', 'x'), ('前', 'f'), ('国脚', 'n'), ('曲波', 'nr'), ('挂帅', 'n'), ('的', 'uj'), ('中国', 'ns'), ('U19B', 'eng'), ('队', 'n'), ('迎战', 'v'), ('泰国', 'ns'), ('U19', 'eng'), (', ', 'x'), ('上半场', 'ns'), ('国青队', 'nt'), ('的', 'uj'), ('门户', 'n'), ('大', 'a'), ('开', 'v'), (', ', 'x'), ('泰国', 'ns'), ('在', 'p'), ('第', 'm'), ('11', 'm'), ('分钟', 'q'), ('和', 'c'), ('第', 'm'), ('17', 'm'), ('分钟', 'q'), ('连进', 'a'), ('2', 'm'), ('球', 'n'), (', ', 'x'), ('半场', 'n'), ('国青', 'nt'), ('0', 'm'), ('射门', 'n'), ('0', 'x'), ('角球', 'n'), (', ', 'x'), ('几乎', 'd'), ('被', 'p'), ('完全', 'ad'), ('压制', 'n'), ('。', 'x'), ('下半场', 'n'), (', ', 'x'), ('国青', 'nr'), ('的', 'uj'), ('进攻', 'v'), ('一度', 'mq'), ('有所', 'n'), ('起色', 'n'), (', ', 'x'), ('并', 'c'), ('由', 'p'), ('马辅渔', 'nr'), ('利用', 'n'), ('远射', 'v'), ('扳回', 'v'), ('一', 'm'), ('球', 'n'), (', ', 'x'), ('但', 'c'), ('最终', 'd'), ('未能', 'v'), ('扳平', 'n'), ('比分', 'd'), ('。', 'x'), ('全场', 'n'), ('比赛', 'vn'), ('结束', 'v'), (', ', 'x'), ('国青', 'nt'), ('1', 'm'), ('-', 'x'), ('2', 'x'), ('输球', 'n'), (', ', 'x'), ('继', 'v'), ('中国', 'ns'), ('杯国', 'n'), ('足', 'a'), ('0', 'm'), ('-', 'x'), ('1', 'x'), ('输给', 'v'), ('泰国', 'ns'), ('之后', 'f'), (', ', 'x'), ('3', 'm'), ('天内', 's'), ('遭遇', 'n'), ('泰国', 'ns'), ('足球', 'n'), ('双', 'm'), ('杀', 'v'), ('。', 'x')]
```

（2）snownlp 命名实体识别结果

```
(nlp) D:\大学\大三下\自然语言处理\实验\exp4>python ex4.py

SnowNLP NER结果：
['3', '月', '23', '日', '下午', ',', '“', '青年报', '杯赛', '”U19', '邀请赛', '在', '越南', '芽', '庄', '进行', ',', ',', '前', '国', '脚曲', '波', '挂帅', '的', '中国', 'U19B', '队', '迎战', '泰国', 'U19,', ',', '上半场', '国', '青队', '的', '门户', '大', '开', ',', ',', '泰国', '在', '第', '11', '分钟', '和', '第', '17', '分钟', '连', '进', '2', '球', ',', ',', '半场', '国', '青', '0', '射门', '0', '角球', ',', ',', '几乎', '被', '完全', '压制', ',', '。', ',', '下半场', ',', ',', '国', '青', '的', ',', '进攻', '一度', '有所', '起色', ',', ',', '并', '由', '马', '辅', '渔', '利用', '远射', '扳回', '一', '球', ',', ',', '但', '最终', '未', '能', '扳平', '比分', '。', '全场', '比赛', '结束', ',', ',', '国', '青', '1-2', '输', '球', ',', ',', '继', '中国', '杯', '国', '足', '0-1', '输', '给', '泰国', '之后', ',', '3', '天', '内', '遭遇', '泰国', '足球', '双杀', '。']
```

（3）THULAC 命名实体识别结果

```
(nlp) D:\大学\大三下\自然语言处理\实验\exp4>python ex4.py
Model loaded succeed

THULAC NER结果：
[['3月', 't'], ['23日', 't'], ['下午', 't'], [', ', 'w'], ['“', 'w'], ['青年', 'n'], ['报杯赛', 'n'], ['”', 'w'], ['U', 'g'], ['19', 'm'], ['邀请赛', 'n'], ['在', 'p'], ['越南', 'ns'], ['芽庄', 'ns'], ['进行', 'v'], [', ', 'w'], ['前', 'f'], ['国脚', 'n'], ['曲波', 'np'], ['挂帅', 'v'], ['的', 'u'], ['中国', 'ns'], ['U19B队', 'n'], ['迎战', 'v'], ['泰国', 'ns'], ['U19', 'x'], [', ', 'w'], ['上半场', 'n'], ['国青队', 'ni'], ['的', 'u'], ['门户', 'n'], ['大', 'd'], ['开', 'v'], [', ', 'w'], ['泰国', 'ns'], ['在', 'p'], ['第11', 'm'], ['分钟', 'q'], ['和', 'c'], ['第17', 'm'], ['分钟', 'q'], ['连', 'd'], ['进', 'v'], ['2', 'm'], ['球', 'n'], [', ', 'w'], ['半', 'm'], ['场', 'q'], ['国青0', 'n'], ['射门', 'v'], ['0', 'v'], ['角球', 'n'], [', ', 'w'], ['几乎', 'd'], ['被', 'p'], ['完全', 'a'], ['压制', 'v'], ['。', 'w'], ['下半场', 'n'], [', ', 'w'], ['国青', 'a'], ['的', 'u'], ['进攻', 'v'], ['一度', 'd'], ['有所', 'v'], ['起色', 'n'], [', ', 'w'], ['并', 'c'], ['由', 'p'], ['马辅渔', 'np'], ['利用', 'v'], ['远射', 'v'], ['扳回', 'v'], ['一', 'm'], ['球', 'n'], [', ', 'w'], ['但', 'c'], ['最终', 'd'], ['未能', 'v'], ['扳平', 'v'], ['比分', 'n'], ['。', 'w'], ['全场', 'n'], ['比赛', 'v'], ['结束', 'v'], [', ', 'w'], ['国青', 'a'], ['1', 'm'], ['-', 'w'], ['2', 'm'], ['输', 'v'], ['球', 'n'], [', ', 'w'], ['继', 'g'], ['中国', 'ns'], ['杯国', 'ns'], ['足0', 'm'], ['-', 'w'], ['1', 'm'], ['输', 'v'], ['给', 'v'], ['泰国', 'ns'], ['之后', 'f'], [', ', 'w'], ['3', 'm'], ['天', 'q'], ['内', 'f'], ['遭遇', 'v'], ['泰国', 'ns'], ['足球', 'n'], ['双杀', 'a'], ['。', 'w']]
```

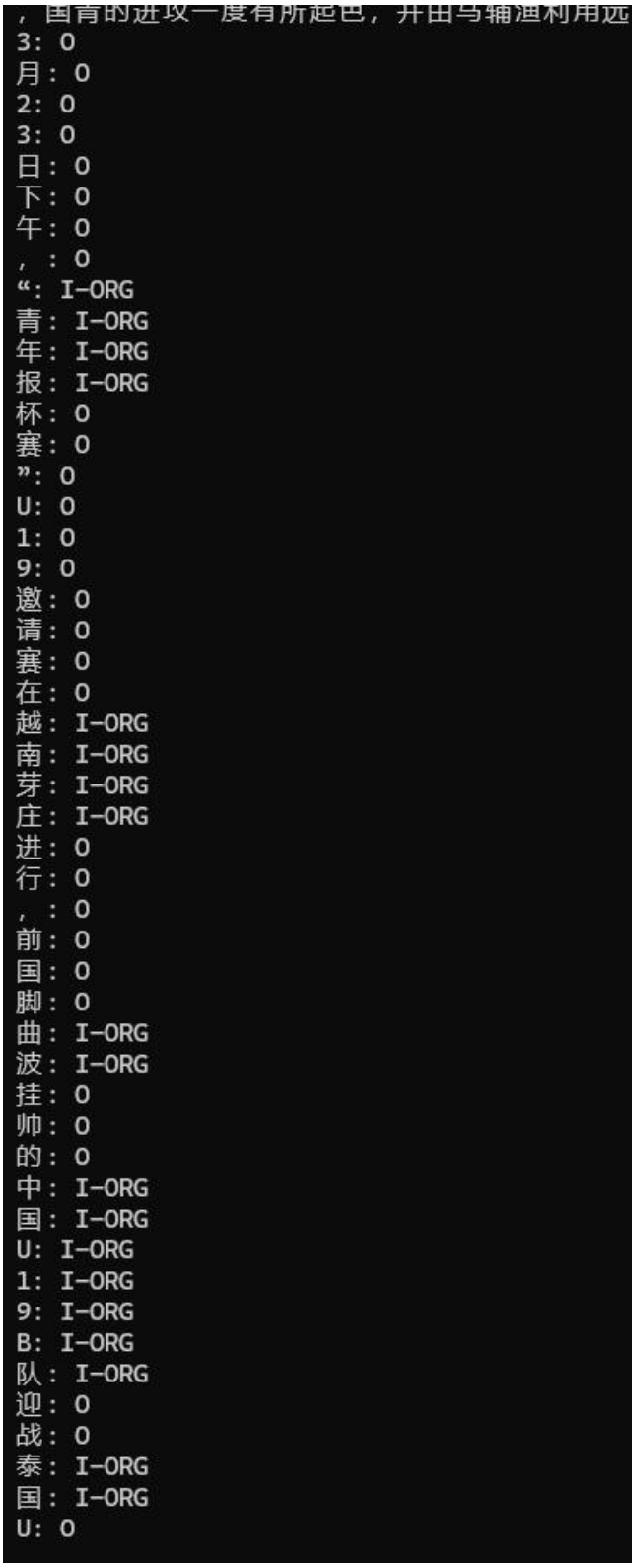
（4）nlpir 命名实体结果

```
(nlp) D:\大学\大三下\自然语言处理\实验\exp4>python ex4.py

NLPIR NER结果：
[('3月', 'time word'), ('23日', 'time word'), ('下午', 'time word'), (', ', 'punctuation mark:comma'), ('“', 'punctuation mark:left quotation mark'), ('青年报', 'multiword expression'), ('杯', 'noun:noun morpheme'), ('赛', 'verb'), ('”', 'punctuation mark:right quotation mark'), ('U19', 'noun'), ('邀请赛', 'noun'), ('在', 'preposition'), ('越南', 'noun:toponym:transcribed toponym'), ('芽', 'noun'), ('庄', 'noun:personal name:Chinese surname'), ('进行', 'verb:performative verb'), (', ', 'punctuation mark:comma'), ('前', 'noun of locality'), ('国脚', 'noun'), ('曲波', 'noun:personal name'), ('挂帅', 'verb:intransitive verb'), ('的', 'particle:particle 的/底'), ('中国', 'noun:toponym'), ('U19B', 'noun'), ('队', 'noun'), ('迎战', 'verb'), ('泰国', 'noun:toponym:transcribed toponym'), ('U19', 'noun'), (', ', 'punctuation mark:comma'), ('上半场', 'noun'), ('国青', 'noun:personal name'), ('队', 'noun'), ('的', 'particle:particle 的/底'), ('门户', 'noun'), ('大', 'adverb'), ('开', 'verb'), (', ', 'punctuation mark:comma'), ('泰国', 'noun:toponym:transcribed toponym'), ('在', 'preposition'), ('第11', 'numeral'), ('分钟', 'classifier:temporal classifier'), ('和', 'conjunction:coordinating conjunction'), ('第17', 'numeral'), ('分钟', 'classifier:temporal classifier'), ('连', 'verb'), ('进', 'verb:directional verb'), ('2', 'numeral'), ('球', 'noun'), (', ', 'punctuation mark:comma'), ('半', 'numeral'), ('场', 'classifier:verbal classifier'), ('国青', 'noun:personal name'), ('0', 'numeral'), ('射门', 'verb:intransitive verb'), ('0', 'numeral'), ('角球', 'noun'), (', ', 'punctuation mark:comma'), ('几乎', 'adverb'), ('被', 'preposition:preposition 被'), ('完全', 'adjective:auxiliary adjective'), ('压制', 'verb'), ('。', 'punctuation mark:period'), ('下', 'verb:directional verb'), ('半', 'numeral'), ('场', 'classifier:verbal classifier'), (', ', 'punctuation mark:comma'), ('国青', 'noun:personal name'), ('的', 'particle:particle 的/底'), ('进攻', 'verb:noun-verb'), ('一度', 'adverb'), ('有所', 'verb'), ('起色', 'noun'), (', ', 'punctuation mark:comma'), ('并', 'conjunction:coordinating conjunction'), ('由', 'preposition'), ('马', 'noun'), ('辅', 'verb:verb morpheme'), ('渔利', 'verb:intransitive verb'), ('用', 'preposition'), ('远射', 'verb'), ('扳回', 'verb'), ('一', 'numeral'), ('球', 'noun'), (', ', 'punctuation mark:comma'), ('但', 'conjunction'), ('最终', 'adverb'), ('未能', 'verb'), ('扳平', 'verb'), ('比分', 'noun'), ('。', 'punctuation mark:period'), ('全场', 'noun'), ('比赛', 'verb:noun-verb'), ('结束', 'verb'), (', ', 'punctuation mark:comma'), ('国青', 'noun:personal name'), ('1-2', 'numeral'), ('输', 'verb'), ('球', 'noun'), (', ', 'punctuation mark:comma'), ('继', 'verb:verb morpheme'), ('中国', 'noun:toponym'), ('杯', 'noun:noun morpheme'), ('国', 'noun'), ('足', 'adjective'), ('0-1', 'numeral'), ('输', 'verb'), ('给', 'preposition'), ('泰国', 'noun:toponym:transcribed toponym'), ('之后', 'noun of locality'), (', ', 'punctuation mark:comma'), ('3', 'numeral'), ('天', 'classifier:temporal classifier'), ('内', 'noun of locality'), ('遭遇', 'verb'), ('泰国', 'noun:toponym:transcribed toponym'), ('足球', 'noun'), ('双', 'numeral'), ('杀', 'verb'), ('。', 'punctuation mark:period')]

(nlp) D:\大学\大三下\自然语言处理\实验\exp4>
```

3、使用 BERT + Bi-LSTM + CRF 实践命名实体识别实验结果



```
国: I-ORG
U: O
1: O
9: O
，: O
上: O
半: O
场: O
国: I-ORG
青: I-ORG
队: I-ORG
的: O
门: O
户: O
大: O
开: O
，: O
泰: O
国: O
在: O
第: O
1: O
1: O
分: O
钟: O
和: O
第: O
1: O
7: O
分: O
钟: O
连: O
进: O
2: O
球: O
，: O
半: O
场: O
国: O
青: O
0: O
射: O
门: O
0: O
角: O
球: O
，: O
几: O
乎: O
被: O
```



实验总结：

可以看到使用已经训练好的集成命名实体识别工具如 Nltk，jieba 等预测结果较为准确。自己使用 BERT+BI-LSTM+CRF 进行训练然后进行预测的效果较差，预测结果多为 O 和 I-ORG。原因推测是因为训练集采用数据较少（因为设备性能限制，只使用了 10000 条数据进行了训练），训练 epoch 较少（epoch=5）且训练集标注质量不够高（训练集标注多为 O）。

附源代码

```
#NLTK
with open('Chinese.txt', 'r', encoding='utf-8') as file:
    chinese_content = file.read()
with open('English.txt', 'r', encoding='utf-8') as file:
    english_content = file.read()

import nltk
from nltk import word_tokenize, pos_tag, ne_chunk
def nltk_ner(text):
    tokens = word_tokenize(text)
    pos_tags = pos_tag(tokens)
```

```
    named_entities = ne_chunk(pos_tags)
    return named_entities
print("NLTK NER 结果:")
print(nltk_ner(english_content))



#spacy
import spacy
nlp = spacy.load("en_core_web_sm")
def spacy_ner(text):
    doc = nlp(text)
    return [(ent.text, ent.label_) for ent in doc.ents]
print("\nSpacy NER 结果:")
print(spacy_ner(english_content))

#stanza

import stanza
# 初始化 pipeline
nlp = stanza.Pipeline('en', processors='tokenize,ner')  # 中文使用 'zh'
def stanza_ner(text):
    doc = nlp(text)
    entities = []
    for sentence in doc.sentences:
        for ent in sentence.ents:
            entities.append((ent.text, ent.type))
    return entities
print("\nStanza (StanfordNLP) NER 结果:")
print(stanza_ner(english_content))

#jieba

import jieba.posseg as pseg
def jieba_ner(text):
    words = pseg.cut(text)
    return [(word, flag) for word, flag in words]
print("\nJieba NER 结果:")
print(jieba_ner(chinese_content))

#thulac



import thulac
```

```
thu = thulac.thulac(seg_only=False)
def thulac_ner(text):
    return thu.cut(text)

print("\nTHULAC NER 结果:")
print(thulac_ner(chinese_content))

#nlpir
import pynlpir
pynlpir.open()
def nlpir_ner(text):
    pynlpir.open()
    segments = pynlpir.segment(text, pos_names='all')
    return segments
print("\nNLPIR NER 结果:")
print(nlpir_ner(chinese_content))
pynlpir.close()

#BERT+BI-LSTM+CRF 由三部分代码组成，分别是 data_loader, model, 以及 train

#data_loader.py

import torch
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizerFast

def load_data(file_path):
    sentences = []
    with open(file_path, 'r', encoding='utf-8') as f:
        tokens, tags = [], []
        for line in f:
            line = line.strip()
            if not line:
                if tokens:
                    sentences.append((tokens, tags))
                    tokens, tags = [], []
            else:
                parts = line.split()
                if len(parts) == 2:
                    tokens.append(parts[0])
                    tags.append(parts[1])
                elif len(parts) == 1: # 有时只有词没有标签，作为 'O' 处理
                    tokens.append(parts[0])
                    tags.append('O')
```

```
        if tokens: # 处理文件末尾没有空行的情况
            sentences.append((tokens, tags))
    return sentences

def get_tags(sentences):
    all_tags = set()
    for _, tags in sentences:
        for tag in tags:
            all_tags.add(tag)

    tag_list = sorted(list(all_tags))
    if "O" not in tag_list:
        tag_list.append("O")

    tag_to_idx = {tag: idx for idx, tag in enumerate(tag_list)}
    idx_to_tag = {idx: tag for tag, idx in tag_to_idx.items()}

    PAD_TAG_ID = tag_to_idx.get("O")

    return tag_to_idx, idx_to_tag, PAD_TAG_ID


class NerDataset(Dataset):
    def __init__(self, sentences, tokenizer, tag_to_idx, max_len,
pad_tag_id):
        self.sentences = sentences
        self.tokenizer = tokenizer
        self.tag_to_idx = tag_to_idx
        self.max_len = max_len
        self.pad_tag_id = pad_tag_id

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        tokens, tags = self.sentences[idx]

        tokenized_inputs = self.tokenizer(
            tokens,
            is_split_into_words=True,
            return_offsets_mapping=True,
            padding='max_length',
            truncation=True,
            max_length=self.max_len,
            return_tensors="pt"
```

```
        )

        word_ids = tokenized_inputs.word_ids(batch_index=0)

        previous_word_idx = None
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(self.pad_tag_id)
            elif word_idx != previous_word_idx:
                label_ids.append(self.tag_to_idx[tags[word_idx]])
            else:
                label_ids.append(self.pad_tag_id)
            previous_word_idx = word_idx

        input_ids = tokenized_inputs["input_ids"].squeeze()
        attention_mask = tokenized_inputs["attention_mask"].squeeze()

        return {
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "labels": torch.tensor(label_ids, dtype=torch.long)
        }
```

#model.py

```
import torch
import torch.nn as nn
from transformers import BertModel
from TorchCRF import CRF

class BertBiLSTMCRF(nn.Module):
    def __init__(self, bert_model_name, num_tags, lstm_hidden_dim,
lstm_layers, dropout_rate):
        super(BertBiLSTMCRF, self).__init__()

        self.bert = BertModel.from_pretrained(bert_model_name)
        self.bert_hidden_dim = self.bert.config.hidden_size

        self.lstm = nn.LSTM(
            input_size=self.bert_hidden_dim,
            hidden_size=lstm_hidden_dim,
            num_layers=lstm_layers,
            bidirectional=True,
```

```
            batch_first=True
        )

        self.hidden2tag = nn.Linear(lstm_hidden_dim * 2, num_tags)

        self.crf = CRF(num_tags, batch_first=True)

        self.dropout = nn.Dropout(dropout_rate)

    def _get_lstm_features(self, input_ids, attention_mask):
        bert_outputs = self.bert(input_ids=input_ids,
attention_mask=attention_mask)
        sequence_output = bert_outputs.last_hidden_state

        sequence_output = self.dropout(sequence_output)

        lstm_out, (hidden, cell) = self.lstm(sequence_output)

        lstm_out = self.dropout(lstm_out)

        emissions = self.hidden2tag(lstm_out)
        return emissions

    def forward(self, input_ids, attention_mask, labels=None):
        """
        计算损失或进行解码
        input_ids: (batch_size, seq_length)
        attention_mask: (batch_size, seq_length), 0 for padding, 1 for real
tokens
        labels: (batch_size, seq_length), 标签 ID 序列
        """
        emissions = self._get_lstm_features(input_ids, attention_mask)

        if labels is not None:
            mask = attention_mask.bool()
            loss = -self.crf(emissions, labels, mask=mask,
reduction='mean')
            return loss
        else:
            mask = attention_mask.bool()
            predictions = self.crf.decode(emissions, mask=mask)
            return predictions
```

```
# train.py

import torch
import torch.optim as optim
from torch.utils.data import DataLoader
from transformers import BertTokenizerFast,
get_linear_schedule_with_warmup
from seqeval.metrics import f1_score, precision_score, recall_score,
classification_report

from data_loader import load_data, get_tags, NerDataset
from model import BertBiLSTMCRF

BERT_MODEL_NAME = 'bert-base-chinese'
TRAIN_FILE = 'train.txt'
DEV_FILE = 'dev.txt'

MAX_LEN = 128
BATCH_SIZE = 16
EPOCHS = 5
LEARNING_RATE = 3e-5
LSTM_HIDDEN_DIM = 256
LSTM_LAYERS = 1
DROPOUT_RATE = 0.1
WEIGHT_DECAY = 0.01

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {DEVICE}")
tokenizer = BertTokenizerFast.from_pretrained(BERT_MODEL_NAME)

train_sentences = load_data(TRAIN_FILE)
dev_sentences = load_data(DEV_FILE)

all_sentences_for_tags = train_sentences + dev_sentences
tag_to_idx, idx_to_tag, PAD_TAG_ID = get_tags(all_sentences_for_tags)
NUM_TAGS = len(tag_to_idx)

print(f"Number of tags: {NUM_TAGS}")
print("Tag to ID mapping:", tag_to_idx)
print("Pad Tag ID for labels (used for subwords/padding):", PAD_TAG_ID,
f"({idx_to_tag[PAD_TAG_ID]})")


train_dataset = NerDataset(train_sentences, tokenizer, tag_to_idx, MAX_LEN,
PAD_TAG_ID)
```

```
dev_dataset = NerDataset(dev_sentences, tokenizer, tag_to_idx, MAX_LEN,
PAD_TAG_ID)

train_dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True)
dev_dataloader = DataLoader(dev_dataset, batch_size=BATCH_SIZE)

model = BertBiLSTMCRF(
    bert_model_name=BERT_MODEL_NAME,
    num_tags=NUM_TAGS,
    lstm_hidden_dim=LSTM_HIDDEN_DIM,
    lstm_layers=LSTM_LAYERS,
    dropout_rate=DROPOUT_RATE
)
model.to(DEVICE)

optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE,
weight_decay=WEIGHT_DECAY)

total_steps = len(train_dataloader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)

def train_epoch(model, data_loader, optimizer, scheduler, device):
    model.train()
    total_loss = 0
    for batch_idx, batch in enumerate(data_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        optimizer.zero_grad()
        loss = model(input_ids, attention_mask, labels)

        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()

        total_loss += loss.item()
```

```
        if (batch_idx + 1) % 5 == 0:
             print(f"  Batch {batch_idx+1}/{len(data_loader)}, Loss:
{loss.item():.4f}")

    return total_loss / len(data_loader)

def evaluate(model, data_loader, device, idx_to_tag, pad_tag_id):
    model.eval()
    all_preds_str = []
    all_labels_str = []

    with torch.no_grad():
        for batch in data_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)

            predictions = model(input_ids, attention_mask)

            for i in range(len(predictions)):
                true_tags_str_sample = []
                pred_sample_tags_str = []

                valid_indices = batch['attention_mask'][i].cpu().tolist()

                pred_tag_ids_for_sample = predictions[i]
                true_label_ids_for_sample =
batch['labels'][i].cpu().tolist()

                idx = 0
                for k in range(MAX_LEN):
                    if valid_indices[k] == 1:
                        if batch['input_ids'][i, k].item() ==
tokenizer.cls_token_id or \
                           batch['input_ids'][i, k].item() ==
tokenizer.sep_token_id:
                            idx +=1
                            continue

                        if true_label_ids_for_sample[k] != pad_tag_id:
                            true_tags_str_sample.append(idx_to_tag[true_l
abel_ids_for_sample[k]])
                            if idx < len(pred_tag_ids_for_sample):
                                pred_sample_tags_str.append(idx_to_tag[pr
ed_tag_ids_for_sample[idx]])
```

```
                            else:
                                pred_sample_tags_str.append(idx_to_tag[pa
d_tag_id])
                        idx += 1
                if true_tags_str_sample:
                    all_labels_str.append(true_tags_str_sample)
                    all_preds_str.append(pred_sample_tags_str)

    if not all_labels_str:
        print("Warning: No valid labels found for evaluation.")
        return 0.0, 0.0, 0.0, ""

    precision = precision_score(all_labels_str, all_preds_str)
    recall = recall_score(all_labels_str, all_preds_str)
    f1 = f1_score(all_labels_str, all_preds_str)
    report = classification_report(all_labels_str, all_preds_str,
digits=4)

    return precision, recall, f1, report
best_f1 = 0.0
for epoch in range(EPOCHS):
    print(f"--- Epoch {epoch+1}/{EPOCHS} ---")

    avg_train_loss = train_epoch(model, train_dataloader, optimizer,
scheduler, DEVICE)
    print(f"Epoch {epoch+1} Training Loss: {avg_train_loss:.4f}")

    precision, recall, f1, report = evaluate(model, dev_dataloader, DEVICE,
idx_to_tag, PAD_TAG_ID)
    print(f"Epoch {epoch+1} Validation: Precision: {precision:.4f}, Recall:
{recall:.4f}, F1: {f1:.4f}")
    print("Validation Classification Report:\n", report)

    if f1 > best_f1:
        best_f1 = f1
        print(f"New best F1: {best_f1:.4f}. Model saved.")

print("Training complete.")
print(f"Best F1 score on Dev set: {best_f1:.4f}")

def predict_sentence(sentence_text, model, tokenizer, idx_to_tag, device,
max_len, pad_tag_id):
    model.eval()
    tokens = list(sentence_text)
```

```
    tokenized_inputs = tokenizer(
        tokens,
        is_split_into_words=True,
        return_offsets_mapping=False,
        padding='max_length',
        truncation=True,
        max_length=max_len,
        return_tensors="pt"
    )

    input_ids = tokenized_inputs["input_ids"].to(device)
    attention_mask = tokenized_inputs["attention_mask"].to(device)

    with torch.no_grad():
        predictions_ids_list = model(input_ids, attention_mask) # list of
lists

    if not predictions_ids_list:
        return []

    pred_tag_ids = predictions_ids_list[0]
    actual_tokens_with_special =
tokenizer.convert_ids_to_tokens(input_ids[0].cpu().tolist())

    results = []
    word_idx_in_original_sentence = 0
    current_pred_idx = 0
    if actual_tokens_with_special[current_pred_idx] ==
tokenizer.cls_token:
        current_pred_idx +=1

    while current_pred_idx < len(pred_tag_ids) and
word_idx_in_original_sentence < len(tokens):
        bert_token = actual_tokens_with_special[current_pred_idx]
        if bert_token == tokenizer.sep_token:
            break
        original_token = tokens[word_idx_in_original_sentence]
        if bert_token.startswith("##"):
            current_pred_idx += 1
            continue
        tag_id = pred_tag_ids[current_pred_idx]
        tag_name = idx_to_tag[tag_id]
        results.append((original_token, tag_name))
```

```
            word_idx_in_original_sentence += 1
            current_pred_idx += 1
    return results
test_sentence = "3 月 23 日下午，“青年报杯赛”U19 邀请赛在越南芽庄进行，前国脚曲波挂帅的中国 U19B 队迎战泰国 U19，上半场国青队的门户大开，泰国在第 11 分钟和第 17 分钟连进 2 球，半场国青 0 射门 0 角球，几乎被完全压制。下半场，国青的进攻一度有所起色，并由马辅渔利用远射扳回一球，但最终未能扳平比分。全场比赛结束，国青 1-2 输球，继中国杯国足 0-1 输给泰国之后，3 天内遭遇泰国足球双杀。"
predicted_tags = predict_sentence(test_sentence, model, tokenizer,
idx_to_tag, DEVICE, MAX_LEN, PAD_TAG_ID)
print(f"\n--- Prediction for: '{test_sentence}' ---")
for token, tag in predicted_tags:
    print(f"{token}: {tag}")
```